Handwritten Digit Recognition

Computer Vision Assignment 2
Astana IT University
Continuation of Assignment 1 with DINOv2 and an interactive demonstration

1 Overview and main findings

This project extends a handwritten digit classifier for classes 0 to 9 using a frozen DINOv2 visual transformer and a trained linear head. The selected DINOv2 variant achieved **100.0% test accuracy** and **1.0000 macro F1** on the saved 100-image test split. Relative to the new baseline CNN run, the accuracy difference was +9.0 percentage points. The improved CNN also achieved 100.0% and was substantially faster. DINOv2 therefore does not establish an accuracy advantage over that stronger CNN on this test set.

The research question is whether a transferable, self-supervised image representation helps on a small digit dataset, and whether additional affine augmentation improves that representation-based classifier. A larger pretrained model is not assumed to be better. All four predefined experimental configurations are reported, including unfavorable results.

Experiment	Test accuracy	Macro F1
Baseline CNN rerun	91.0%	0.9078
Improved CNN rerun	100.0%	1.0000
DINOv2 without augmentation	99.0%	0.9900
DINOv2 with augmentation	100.0%	1.0000

Table 1. Measured A2 results on the same saved split. The application selects **DINOv2 with augmentation** by validation cross-entropy before test evaluation. Scores measure a ten-class classification task; mAP and IoU are not applicable.

Relationship to Assignment 1

A1 reported 90% test accuracy and macro F1 0.8969 for its baseline CNN, and 98% accuracy and macro F1 0.9794 for its improved CNN. Those are historical TensorFlow runs. The new CNNs repeat the architectures in PyTorch with a fixed seed; they are not the original fitted weights. Initialization, ordering and augmentation implementation can change the result. The exact existing train/validation/test folders are retained for A2.

Scope of the modern method

DINOv2 is used under the assignment's option to choose an alternative modern SOTA method [1,2]. This is a foundation-model transfer experiment, not a claim of a new state of the art on digits or of using the newest model available in 2026. The project does not implement ArtAug, TA-TiTok or MaskGen.

2 Dataset and augmentation

The dataset contains 1,000 processed grayscale images, 100 for each digit. Half are self-collected and half are downloaded. Every image is 28 by 28 pixels. The audit successfully decoded all files and found no pixel-identical duplicates within or across splits. File names, labels, source categories and pixel SHA-256 hashes are saved in dataset_manifest.json. This audit does not rule out near-duplicates or shared writers.

Split	Own	Downloaded	Total	Per class
Train	400	400	800	80
Validation	50	50	100	10
Test	50	50	100	10

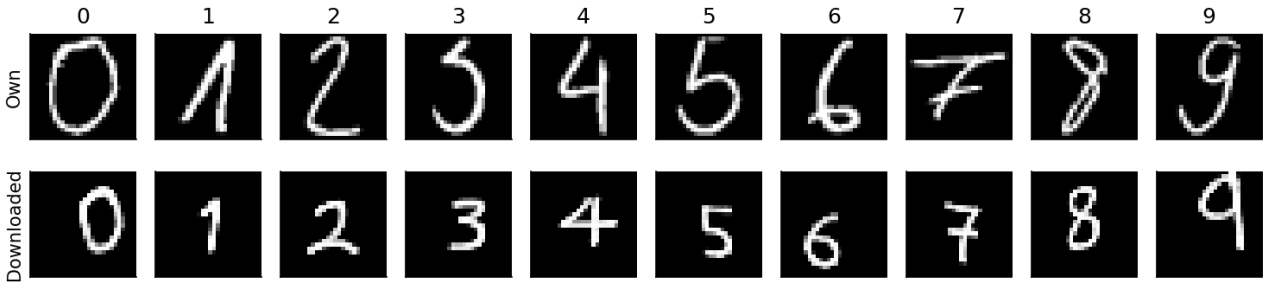


Figure 1. Actual training examples, one own and one downloaded image per digit.

A1 processed own photographs with grayscale conversion, inverse Otsu thresholding, the largest external contour, a square canvas with four pixels of padding on each side, and area resizing. Downloaded RGBA images used the alpha channel; other images used grayscale conversion and resizing. A2 trains on these existing processed images, avoiding changes to the comparison split.

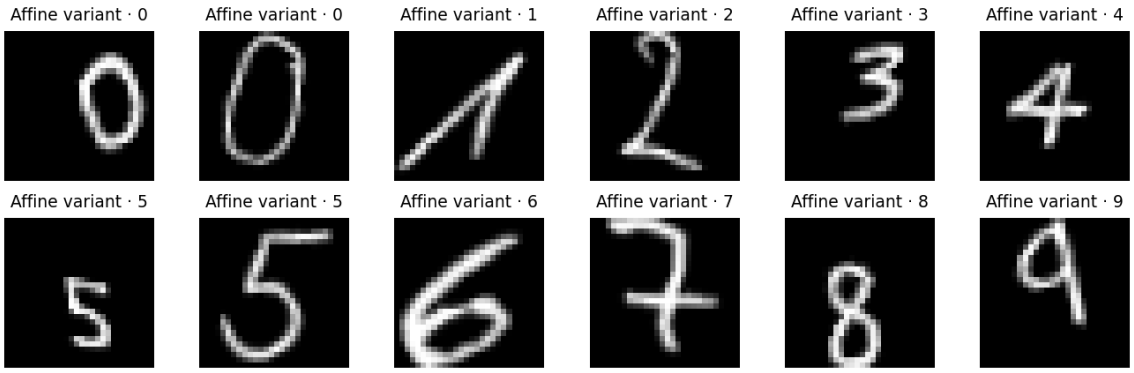


Figure 2. Affine training variants, labeled by digit. These are transformed source images, not generative-model outputs.

The DINOv2 augmentation arm adds two variants per training original: rotation within ±10 degrees, translation within ±2 pixels and scale within 0.92 to 1.08, with black borders and bilinear interpolation. The pool is 800 originals plus 1,600 variants: 66.7% transformed samples and 0% generative-model images. No flipping is used because it can change digit identity. Validation and test are not augmented. The ablation compares this treatment with the original 800-image pool.

3 How the DINOv2 method works

Why transfer visual features

A small supervised CNN must learn useful image features from the 800 labeled training examples. DINOv2 instead provides a representation learned beforehand on a large and diverse image collection [1]. A new task can use that representation and train a much smaller classifier. This may reduce the amount of task-specific learning required, although natural-image pretraining can also mismatch simple grayscale handwriting.

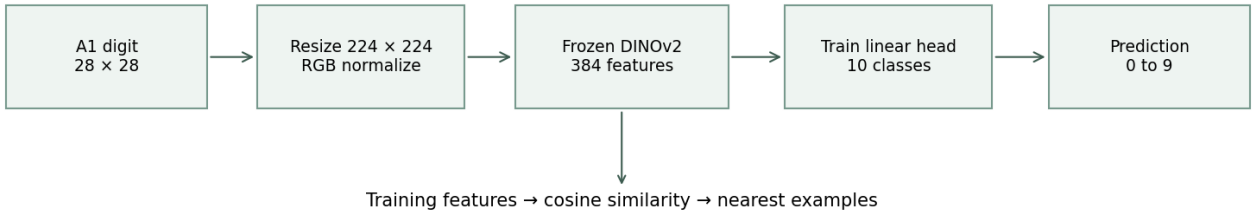


Figure 3. Implemented A2 pipeline. The transformer is frozen; only the ten-class linear head is trained.

The self supervision idea

The DINO family uses teacher and student networks to learn from different views of an image without ordinary class labels. Neurohive's accessible explanation of the original DINO describes this view-matching idea [4]. DINOv2 develops a broader pretrained feature-learning system [1]. The Neurohive article is background on DINO from 2021, not an announcement of DINOv2. This assignment reuses published pretrained weights; it does not reproduce the large-scale self-supervised pretraining.

What is implemented here

The official facebook/dinov2-small checkpoint [3] uses a ViT-S/14 backbone. Each digit is resized to 224 by 224 without cropping, copied into three RGB channels, and normalized with ImageNet channel means and standard deviations. Patches enter the transformer, whose CLS token supplies a 384-value image representation. The backbone runs in evaluation mode with gradients disabled.

Feature means and standard deviations are computed from the original training set only. A linear layer maps standardized features to ten logits; softmax converts them to relative class scores. AdamW updates this head with learning rate 0.01 and weight decay 0.01. Full-batch updates make the number of optimization steps comparable across ablation arms. The maximum is 200 epochs, with patience 20 and restoration of the lowest-validation-loss weights.

Why the explanation is useful

The application compares a query representation with saved training representations using cosine similarity. The closest five images form a visual reference gallery. This can reveal shape ambiguity or source differences. Retrieval is separate from the linear classifier and never changes its prediction. It is not a causal saliency map, and neighbors shown for a CNN query are still DINOv2 neighbors rather than CNN internal explanations.

4 Experiment setup and training accuracy

Model	Architecture and training
Baseline CNN	Conv16 valid 3×3 → pool → Dense32 → 10 logits; Adam 0.001; 15 epochs.
Improved CNN	Conv32 same → pool → Conv64 same → pool → dropout 0.25 → Dense64 → dropout 0.30 → 10 logits. Adam 0.001; at most 30 epochs; patience 5.
DINOv2 heads	Frozen 384-feature backbone → standardization → linear 10-class head. Same seed and optimizer in both augmentation arms.

All runs use seed 42, batch size 32 for CNN training, and sparse class labels with cross-entropy. The improved CNN receives A1-style affine augmentation with rotations up to 28.8 degrees, shifts up to 2.24 pixels and scale changes of up to 8%; its implementation uses OpenCV black borders rather than TensorFlow's default transform filling. The baseline uses its final epoch to match the A1 stopping rule; the improved CNN restores its best validation epoch.

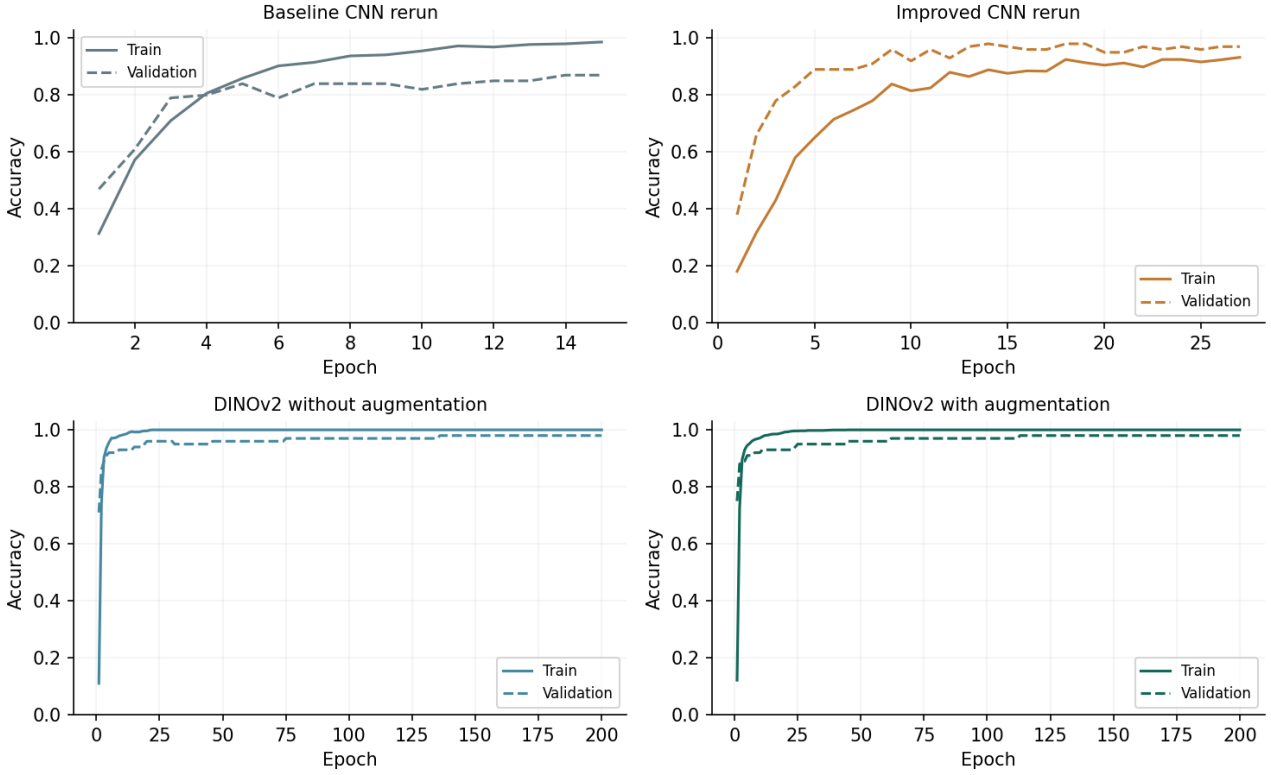


Figure 4. Actual training and validation accuracy histories. Training accuracy is measured with augmentation/dropout active where applicable; validation uses evaluation mode.

The training set drives parameter updates. Validation determines early stopping and which of the two DINOv2 heads is used in the app. Test labels are used only for final reporting of the four predefined configurations after selection. There was no test-based search over architectures, augmentation strengths or head hyperparameters in this A2 run.

5 Loss curves and augmentation ablation

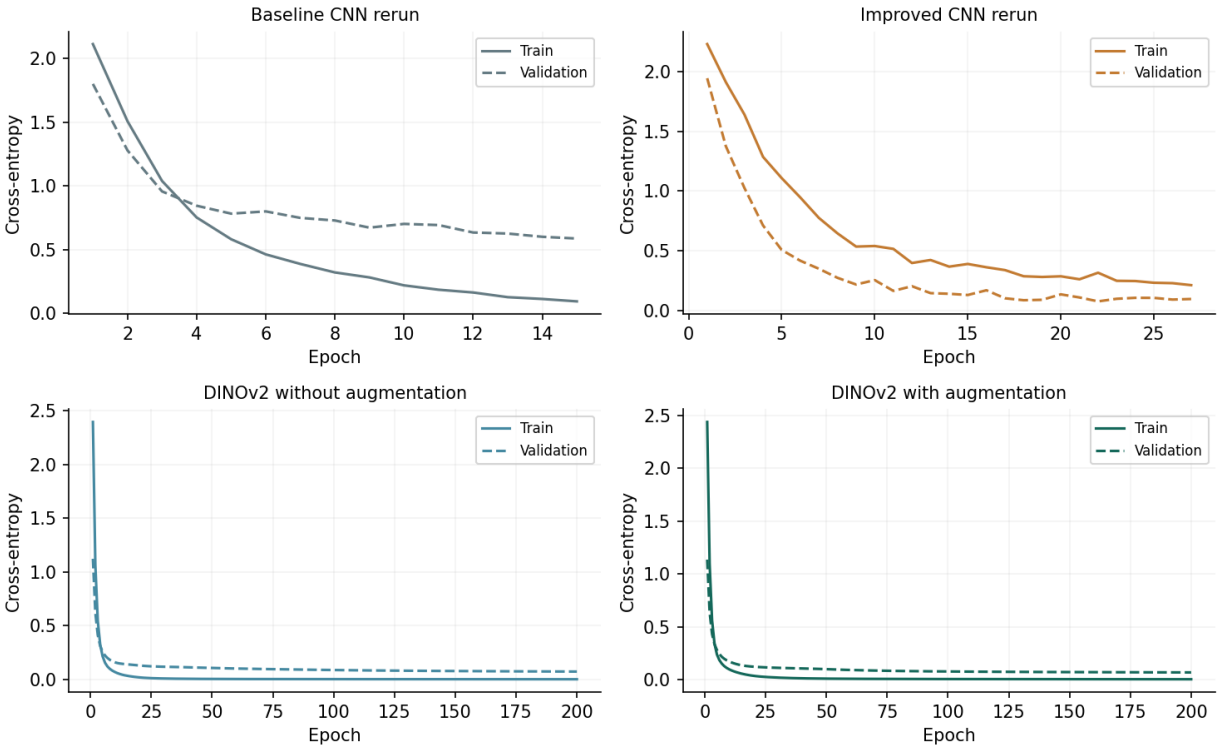


Figure 5. Cross-entropy histories from the actual runs. A low training loss alone is not evidence of good generalization.

DINOv2 variant	Val loss	Test accuracy	Macro F1
DINOv2 without augmentation	0.0726	99.0%	0.9900
DINOv2 with augmentation	0.0665	100.0%	1.0000

Adding the 1,600 affine variants changed test accuracy by **+1.0 percentage points** and macro F1 by **+0.0100**. The validation-selected model was DINOv2 with augmentation. Both arms shared the encoder, seed, head initialization, original-training feature standardization, optimizer, full-batch update policy, early-stopping rule and validation set. The changed treatment was the additional transformed training samples.

This ablation measures the practical effect of the chosen augmentation treatment, including its larger sample pool. It does not isolate every possible factor or prove that all augmentation is useful. With only one seed, some differences may reflect optimization variation. More seeds and an additional writer-held-out dataset would be required for a stronger conclusion.

The selected model classified 100 of 100 test images correctly. A 95% Wilson interval for its accuracy is approximately 96.3% to 100.0%, under an independent-sample assumption. Shared writers or image sources may violate that assumption. A one-image difference equals one percentage point here; small score changes should not be overinterpreted.

6 Error analysis and visual explanation

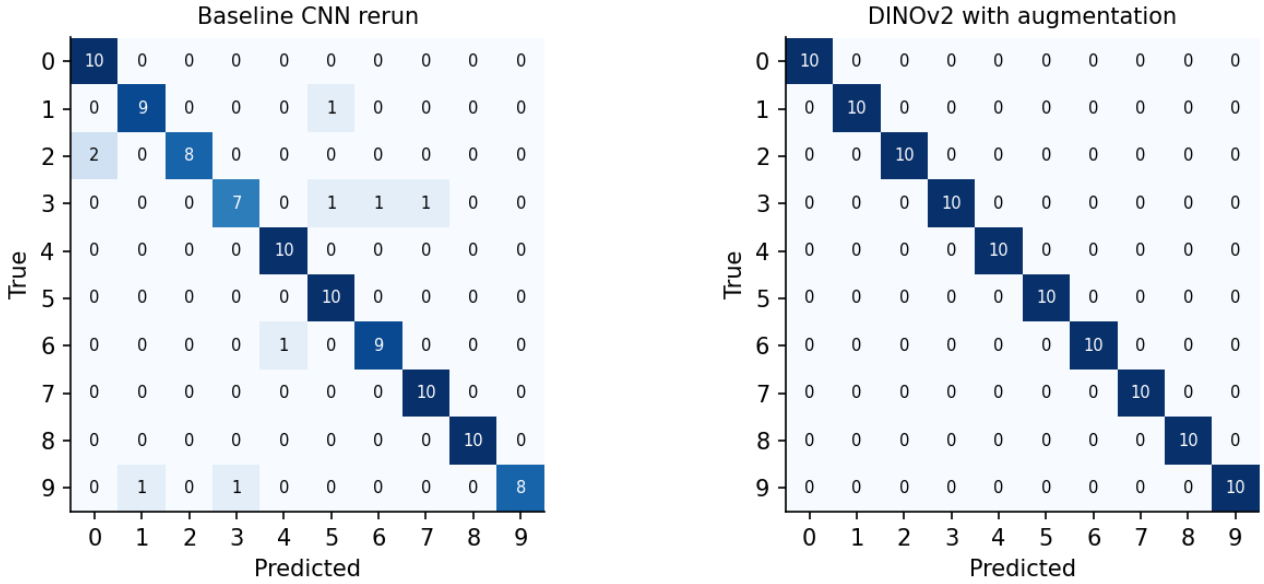


Figure 6. Test confusion matrices for the new baseline CNN and validation-selected DINOv2 model. Rows are true digits; columns are predictions.

All ten classes have precision, recall and F1 equal to 1.00 in this selected model's test run: no errors were observed among the 100 examples. This does not establish perfect recognition outside the sample. Each class has only ten images. The unaugmented DINOv2 arm made one error, so the augmentation benefit is just one corrected test image. Full reports for every model are saved in metrics.json.

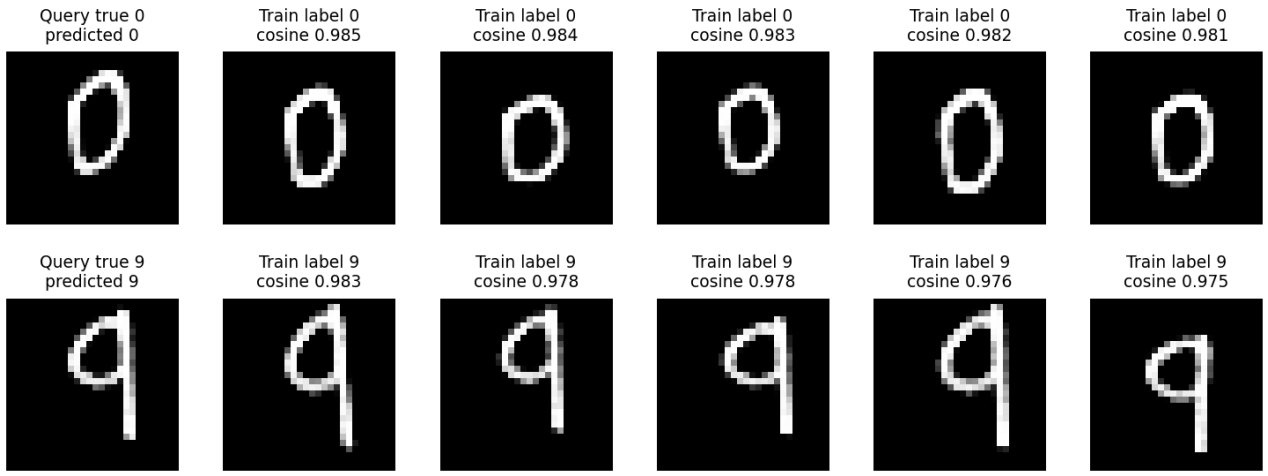


Figure 7. Actual test queries and their five nearest training images in frozen DINOv2 feature space. Errors are shown first when available; remaining rows use fixed examples.

The gallery supports inspection of whether a query resembles correct-class or competing-class training images. A correct neighbor does not guarantee a correct classifier output. Conversely, a wrong neighbor does not prove a bad prediction. Cosine similarity is not a calibrated probability. Test examples are never inserted into the retrieval bank or used for live retraining.

7 Interactive application and latency

Digit Lab is a local Flask web application. Its core flow is draw/upload → prepare the digit → choose CNN or DINOv2 → predict → inspect the processed input, class scores and nearest training examples. Preparation is automatic: opaque grayscale 28×28 inputs with a dark border preserve their pixels; other inputs use photo preprocessing. Photo/drawing mode chooses polarity from the image border and applies thresholding, largest-contour cropping and padding. Only one digit on a simple background is supported.

Model	Mean ms	Median ms	P95 ms
Baseline CNN rerun	0.26	0.25	0.27
Improved CNN rerun	0.51	0.47	0.59
DINOv2 without augmentation	104.86	106.78	114.06
DINOv2 with augmentation	99.48	101.72	113.79

Table 4. Thirty batch-one measurements per model after five warm-ups. Execution used CPU with 4 PyTorch CPU threads. Timings include tensor conversion and model forward computation; DINOv2 also includes resizing and channel normalization. They exclude disk reads, upload, image decoding and nearest-neighbor retrieval. Values are hardware-dependent, not browser-only inference speeds.

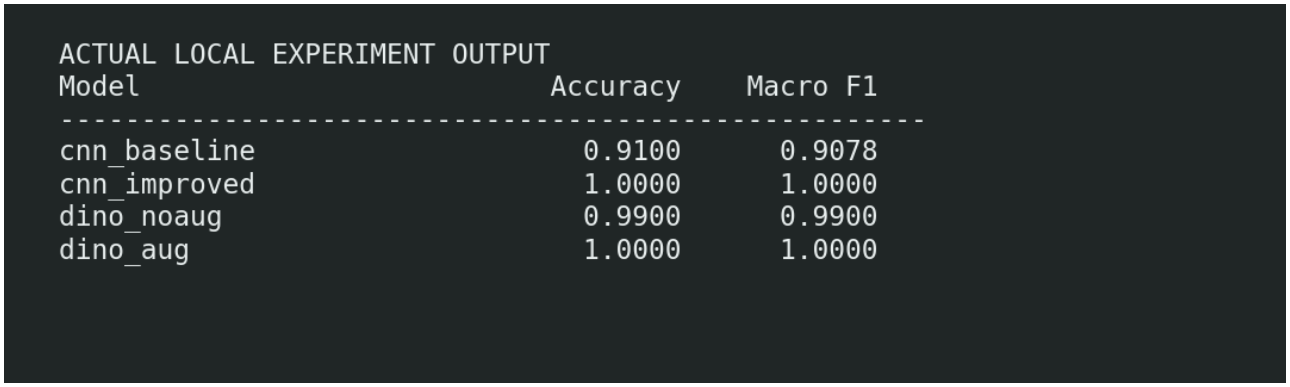


Figure 8. Rendered text output of the measured local experiment, generated directly from metrics.json. This is an output figure, not a browser screenshot.

The page shows the last prediction time and a per-model session average. Detailed software information is retained in artifacts/environment.json. The session average may contain first-call overhead; it is distinct from the warm benchmark above. The app loads model weights once, processes images in memory and does not save uploaded images. The server binds to 127.0.0.1 and does not publish a public endpoint.

Demonstration and operational limits

For the 3–5 minute demonstration, show the A2 notebook boundary, one prepared test prediction, one drawing or photo, the neighbor gallery, latency, the ablation table and the repository. A separate Russian guide provides timings and an English speaking script. Recording and uploading the demo video remain the student's task. Blank photo inputs are rejected, but the classifier is not trained to reject every unknown symbol; letters and multiple digits can receive confident wrong predictions.

8 Ethics limitations and reproducibility

The collection contains self-collected and downloaded handwriting. Available A1 materials do not establish the downloaded source URLs, licenses or consent records; verify these before redistribution. Original data and cached training images are excluded from public Git. No face synthesis or identity recognition is performed. Affine variants are disclosed as transformed data, not independent human samples. Uploaded application inputs remain on the local server in memory.

The balanced class counts do not remove bias in handwriting style, writer identity, pen type, lighting or downloaded source. The split is image-level, not demonstrably writer-disjoint. A1 exploration already exposed test performance, so this continued evaluation is not an untouched external benchmark. The small sample size, single seed, preprocessing sensitivity and natural-image-to-digit domain shift limit generalization. A suitable next study would collect new consented writers and repeat the predefined protocol across several seeds.

How to reproduce and run

The measured environment used Python 3.12.14, PyTorch 2.6.0+cpu, Transformers 4.48.3, NumPy 2.1.3 and scikit-learn 1.6.1. The exact platform, timestamp and parameters are saved in artifacts/environment.json and experiment_config.json. Downloaded DINOv2 weights are identified by their source revision and SHA-256 provenance file. Random seed: 42.

Create a Python 3.12 virtual environment, install requirements.txt, and run `python digit_pipeline.py --data "PATH/ass comp vision"`. Then run `python make_figures.py` and `python app.py`. Open http://127.0.0.1:7860 in Chrome. The complete local bundle includes weights and the feature bank; from Git alone, supply the A1 split and run training to recreate excluded data-dependent artifacts. README.md gives full commands. The combined notebook preserves A1 and provides a separate Colab-compatible A2 workflow.

Repository: https://github.com/silence99999/comp_vision_2

References

[1] Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. arXiv:2304.07193, 2023; revised 2024. https://arxiv.org/abs/2304.07193

[2] Meta AI / FAIR. Official DINOv2 code and model documentation. https://github.com/facebookresearch/dinov2

[3] facebook/dinov2-small. Official converted checkpoint and model card. https://huggingface.co/facebook/dinov2-small

[4] Shevchenko, P. FAIR developed DINO architecture for self-supervised image analysis. Neurohive, 3 May 2021. Background on original DINO. https://neurohive.io/en/novosti/dino-en/